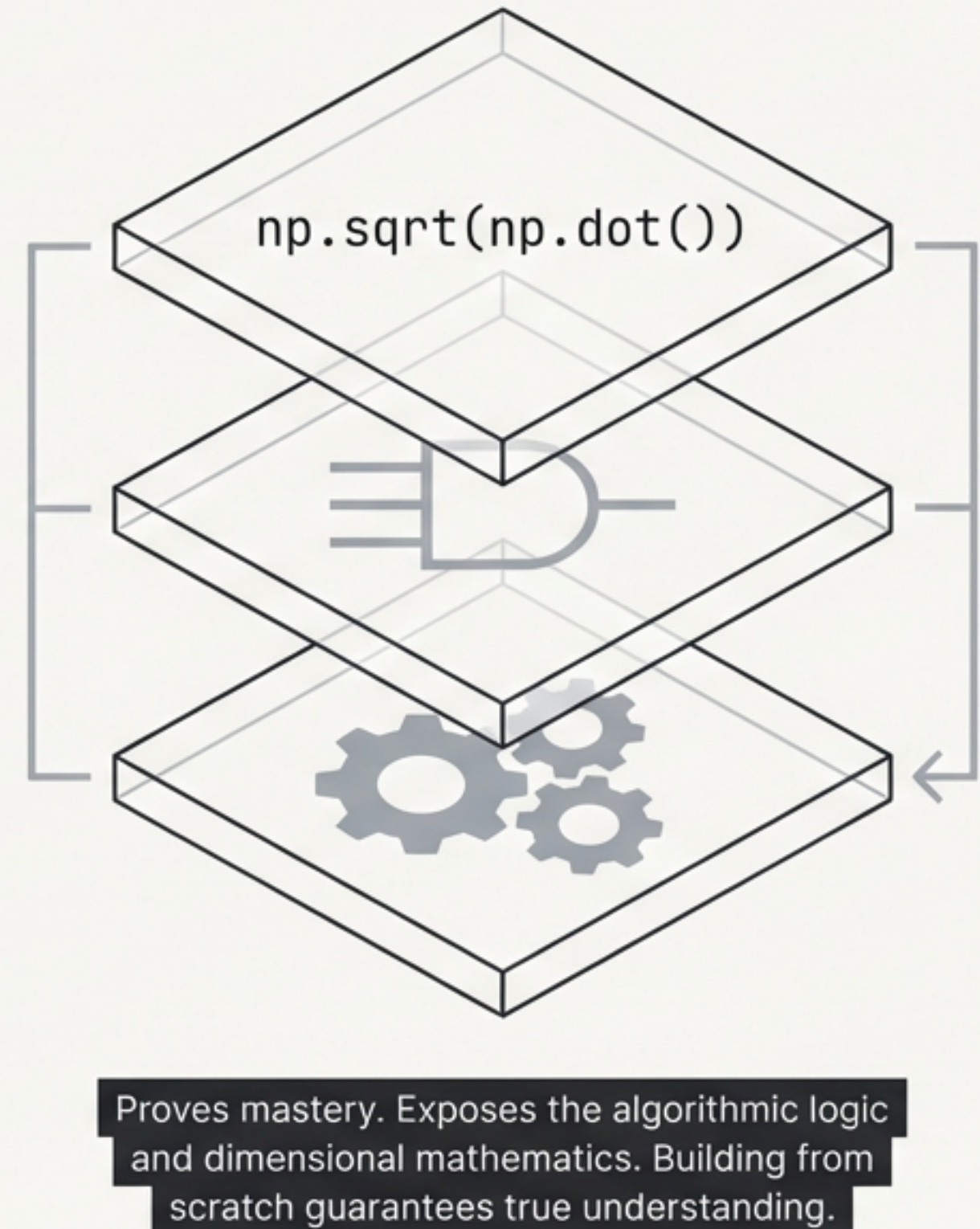
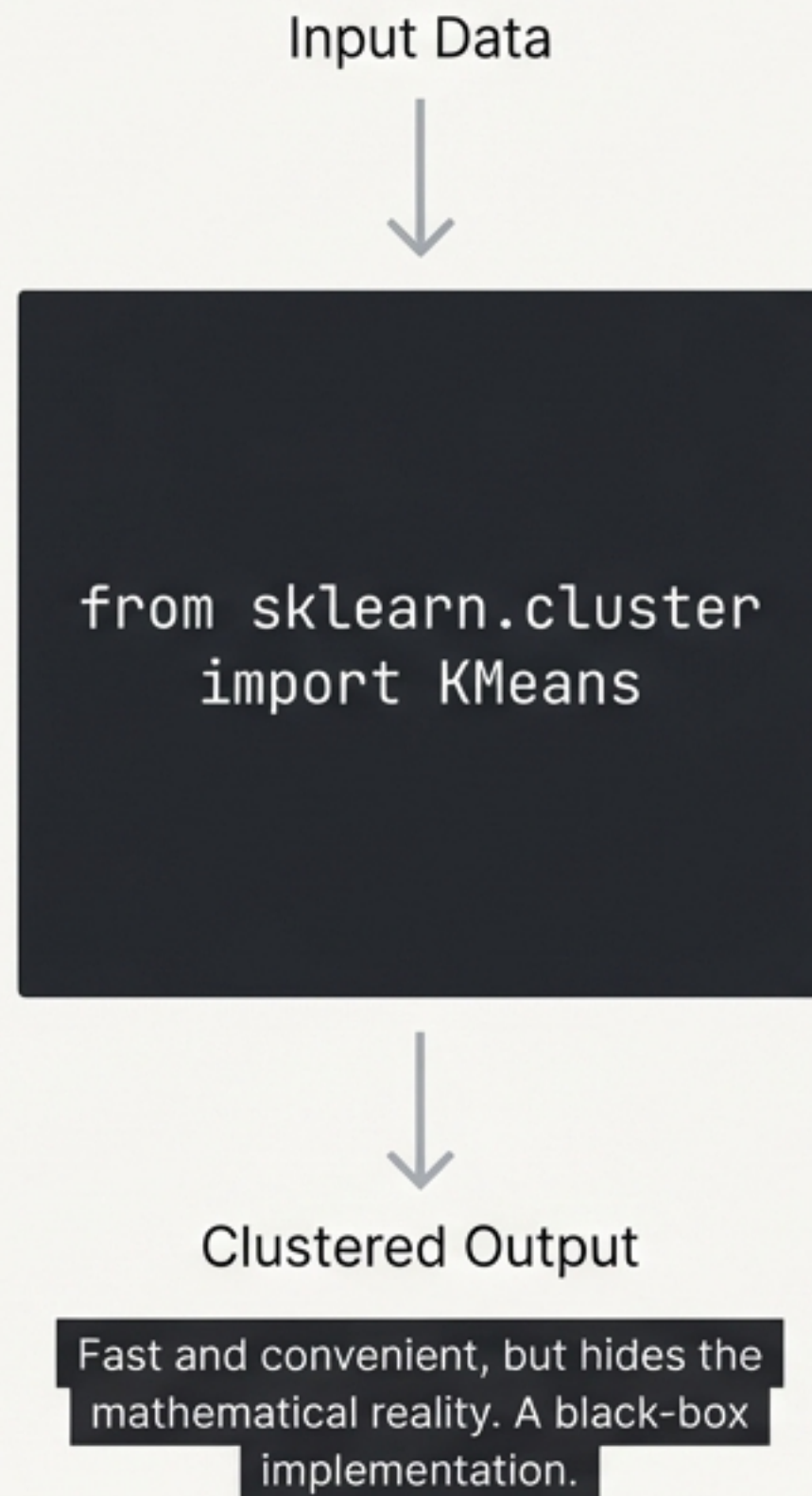




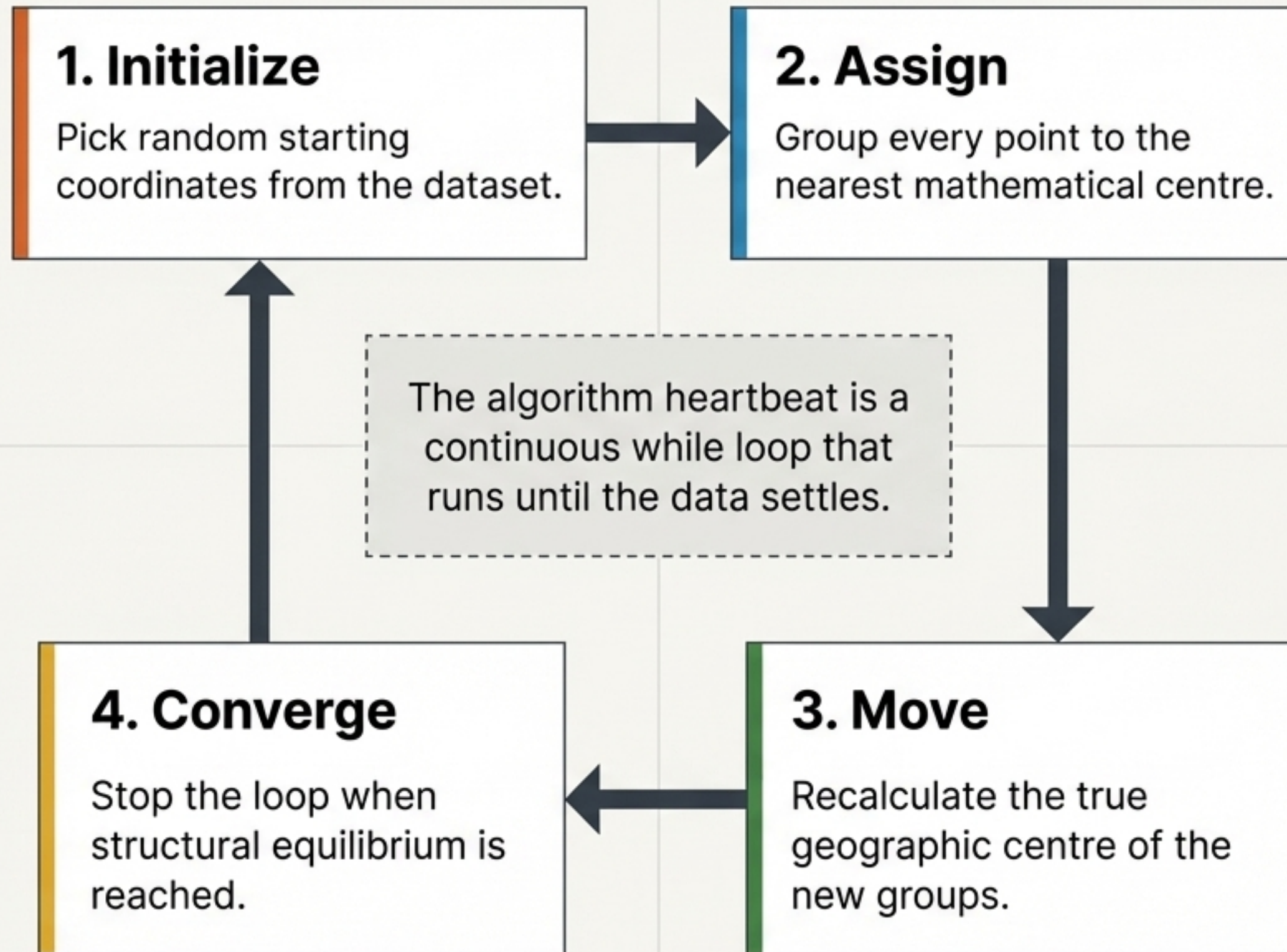
K-Means Clustering From Scratch

Deconstructing the machine learning algorithm in Python

Moving from Algorithm User to Algorithm Engineer



The Algorithm Lifecycle



Step 1: Establishing the Class and Random State

The Code

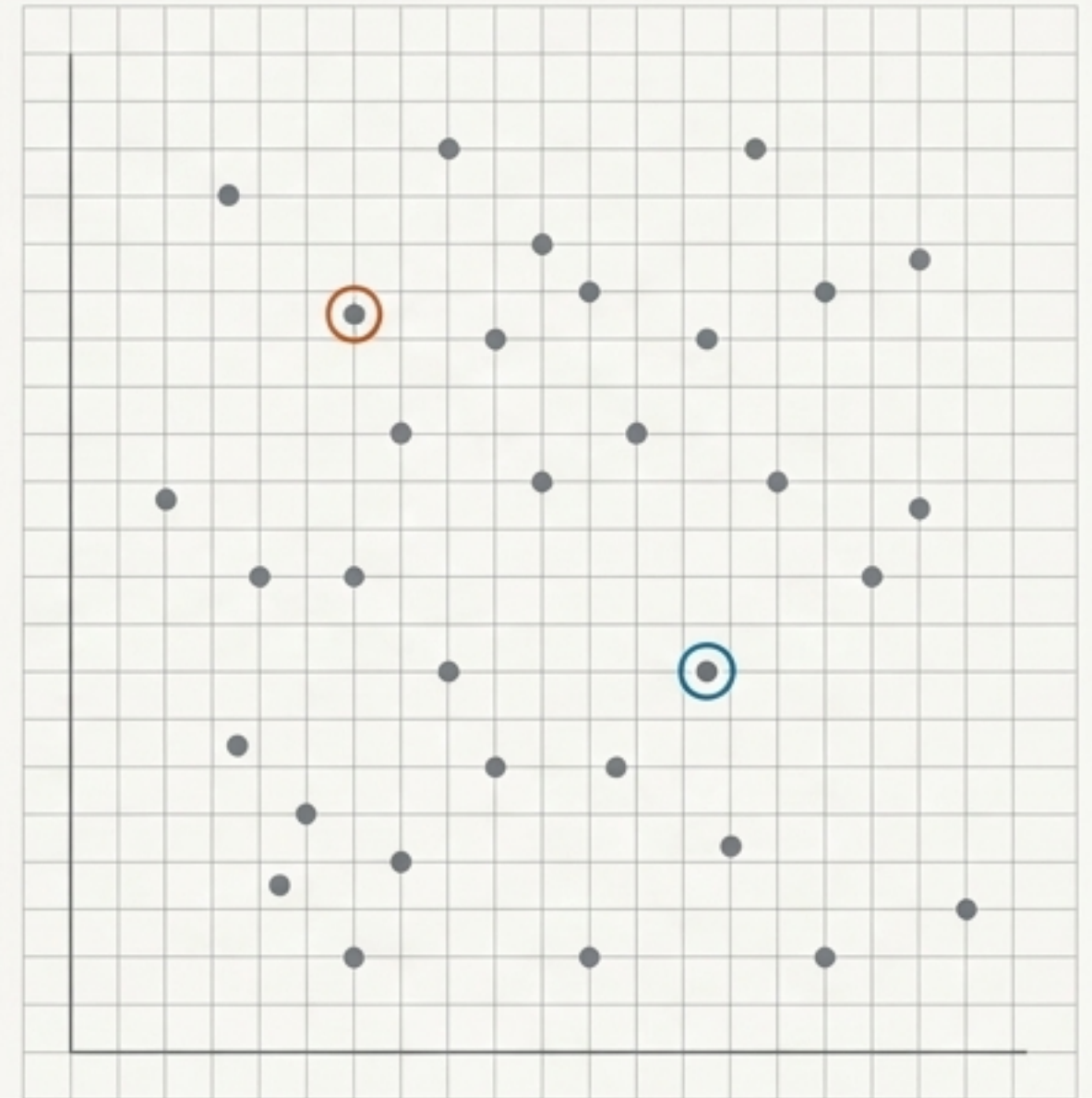
```
class KMeans:
    def __init__(self, n_clusters=2, max_iter=100):
        self.n_clusters = n_clusters
        self.max_iter = max_iter

    def fit(self, X):
        random_indices = random.sample(
            range(0, X.shape[0]),
            self.n_clusters
        )
```

Defines the number of target centroids.

Selects existing data points to guarantee starting centroids are within the actual distribution.

The Concept



The Math Interlude: N-Dimensional Distance

The 2D Trap

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

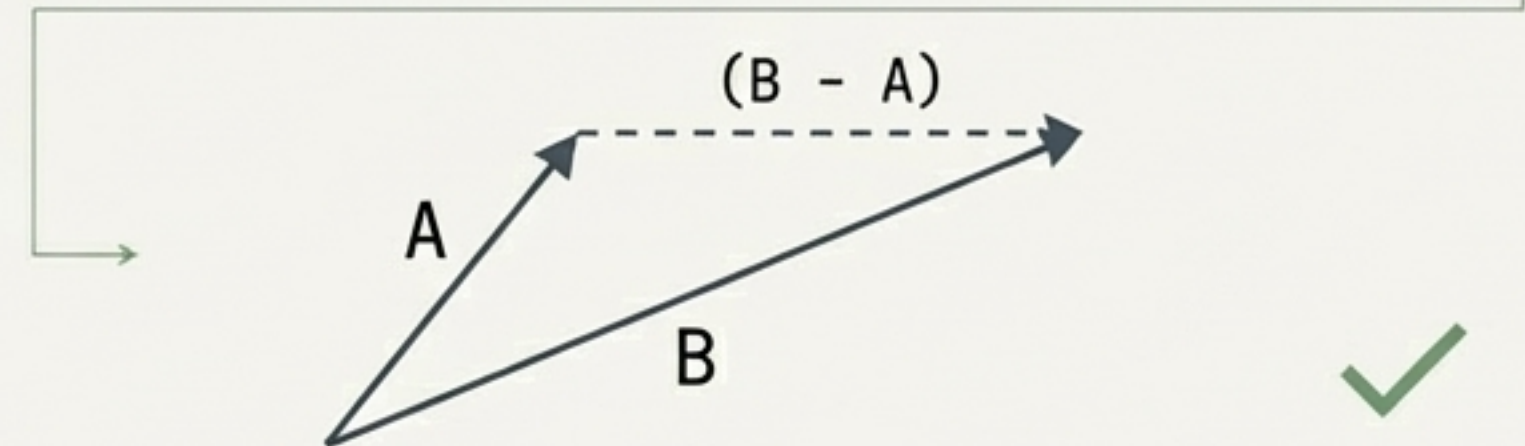
$$+ (z_2 - z_1)^2 + (a_2 - a_1)^2 \dots$$
$$+ \dots$$



Hardcoding geometric formulas fails when a dataset shifts from 2 to 100 features. The equation breaks at scale.

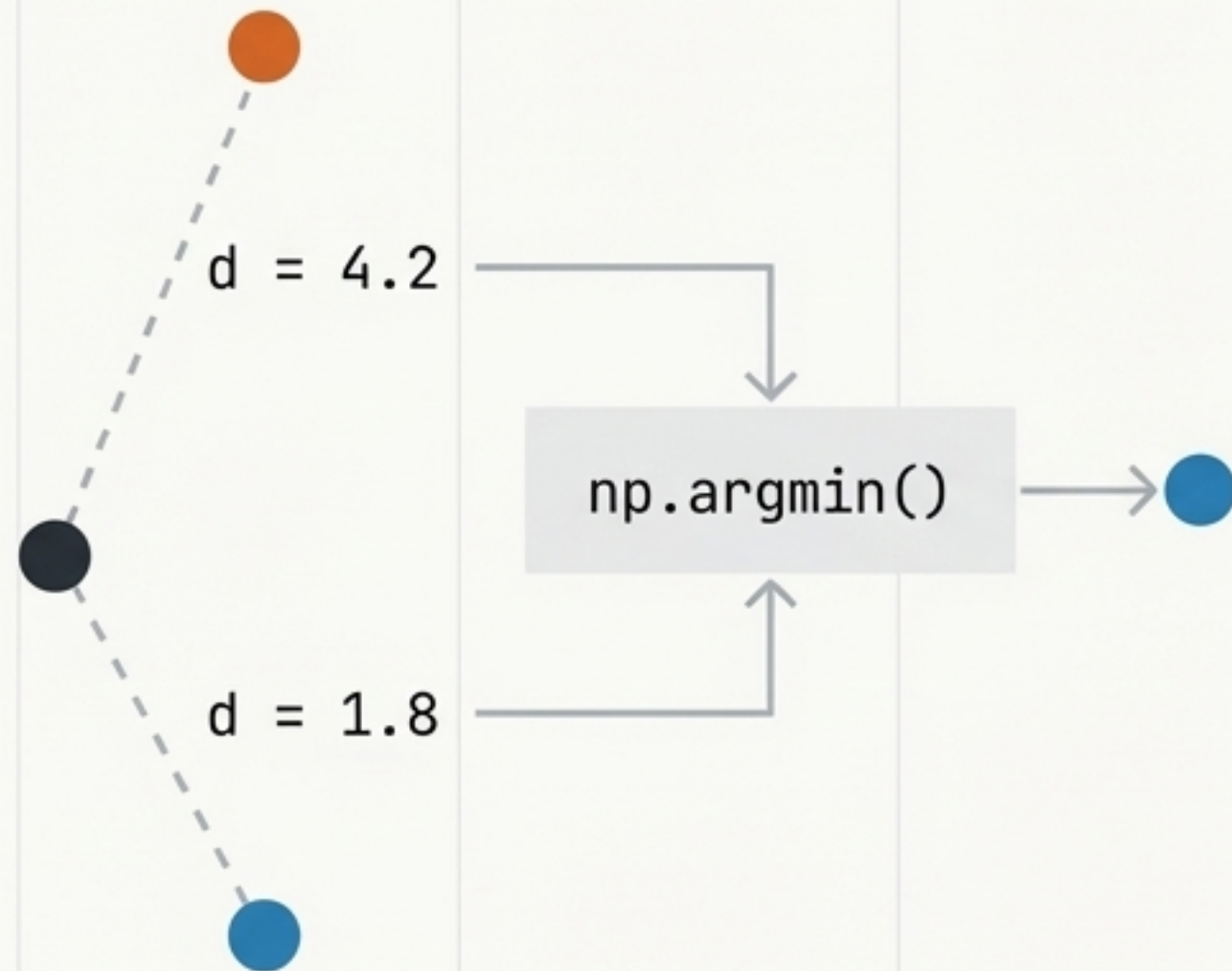
The N-Dimensional Solution

```
np.sqrt(np.dot(X - Y, X - Y))
```



Treating coordinates as vectors and calculating the dot product elegantly handles infinite dimensions without changing a single line of Python.

Step 2: Assigning Points to the Nearest Cluster



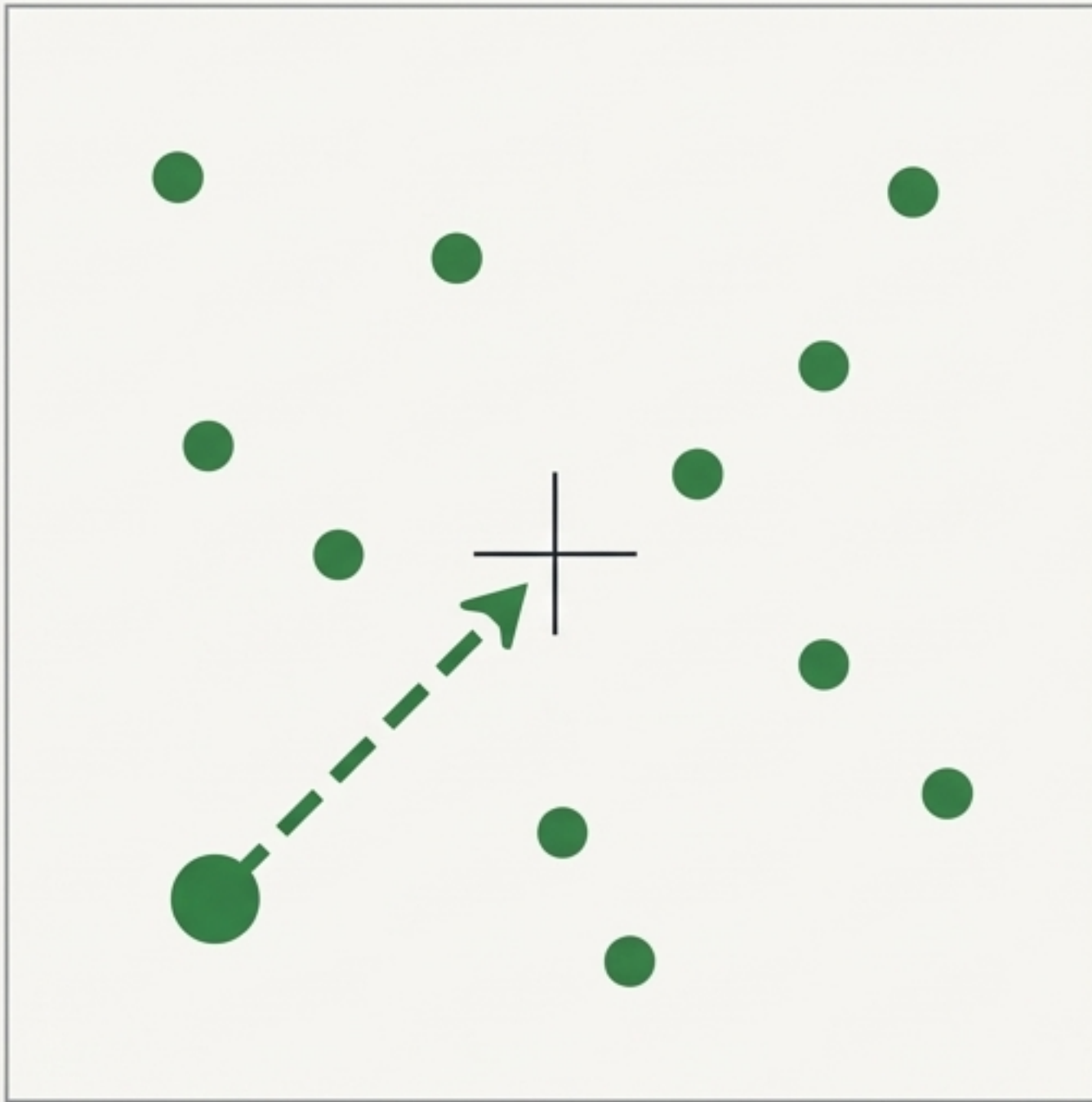
The Assignment Funnel

```
distances = []
for centroid in self.centroids:
    dist = np.sqrt(np.dot(row - centroid, row - centroid))
    distances.append(dist)

cluster_group.append(np.argmin(distances))
```

The algorithm calculates Euclidean distance to all active centroids. The index of the minimum distance mathematically dictates the new cluster assignment.

Step 3: Recalculating the Centroid Positions



1. Isolate the specific cluster

```
X[cluster_group == type]
```

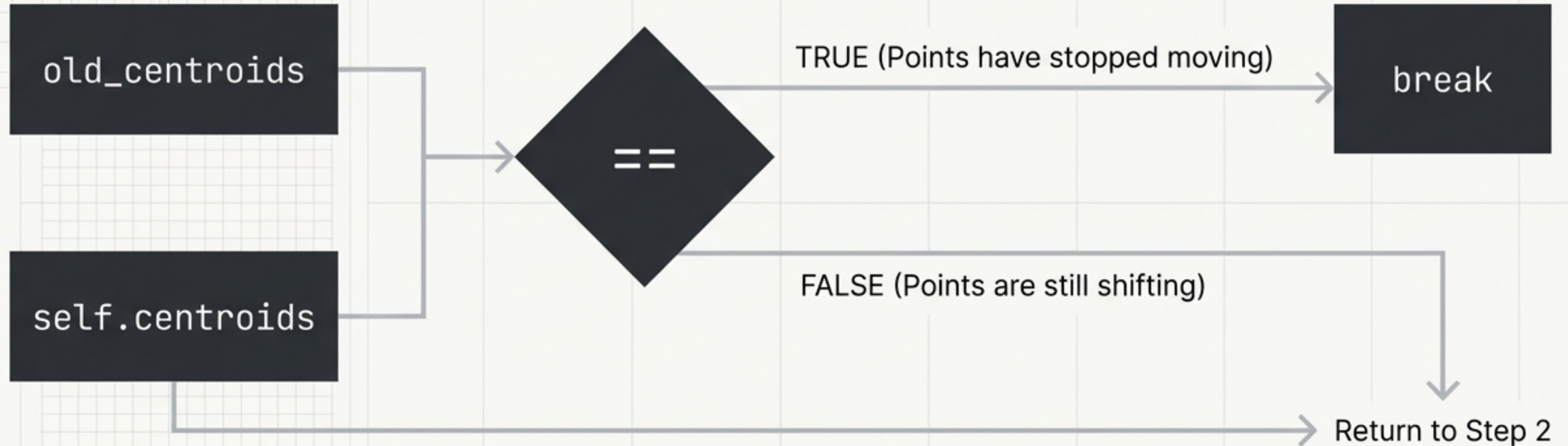
2. Calculate the mean across the axes

```
np.mean(..., axis=0)
```

3. Update the exact centre of gravity

```
new_centroids.append(cluster_mean)
```


Step 4: Checking for Mathematical Convergence

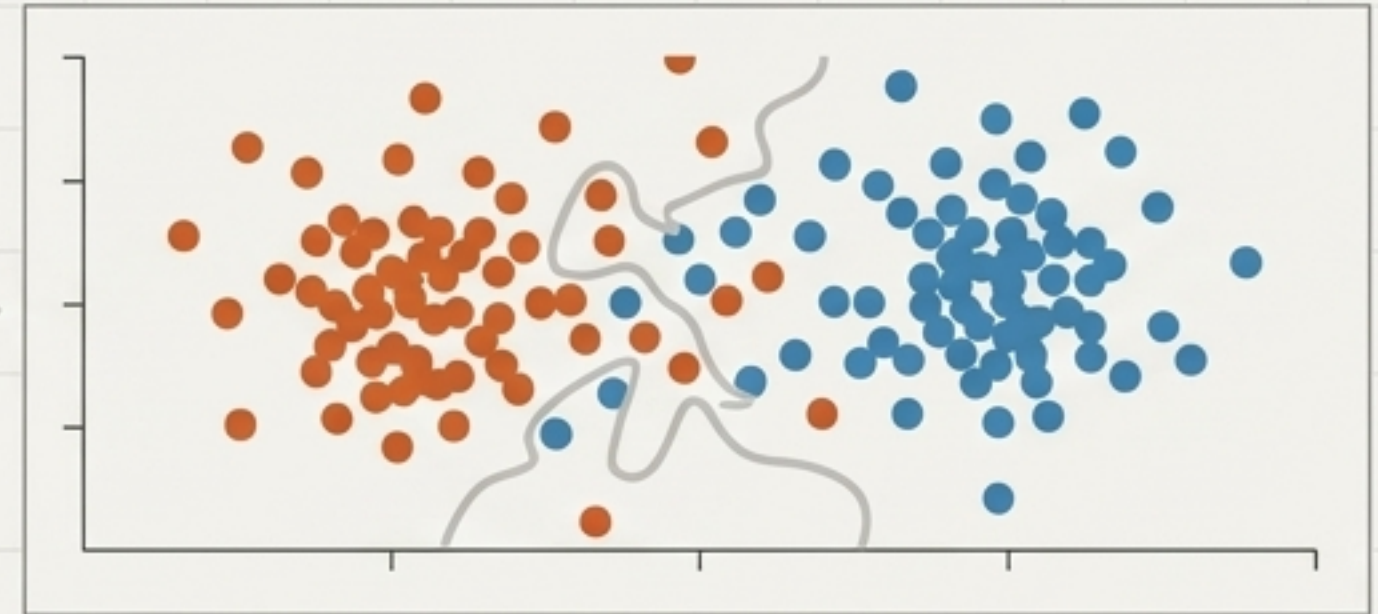


If the mathematical centres of gravity do not change between iterations, structural equilibrium is reached. The loop breaks early to save compute power.

The Impact of Iterations on Cluster Accuracy

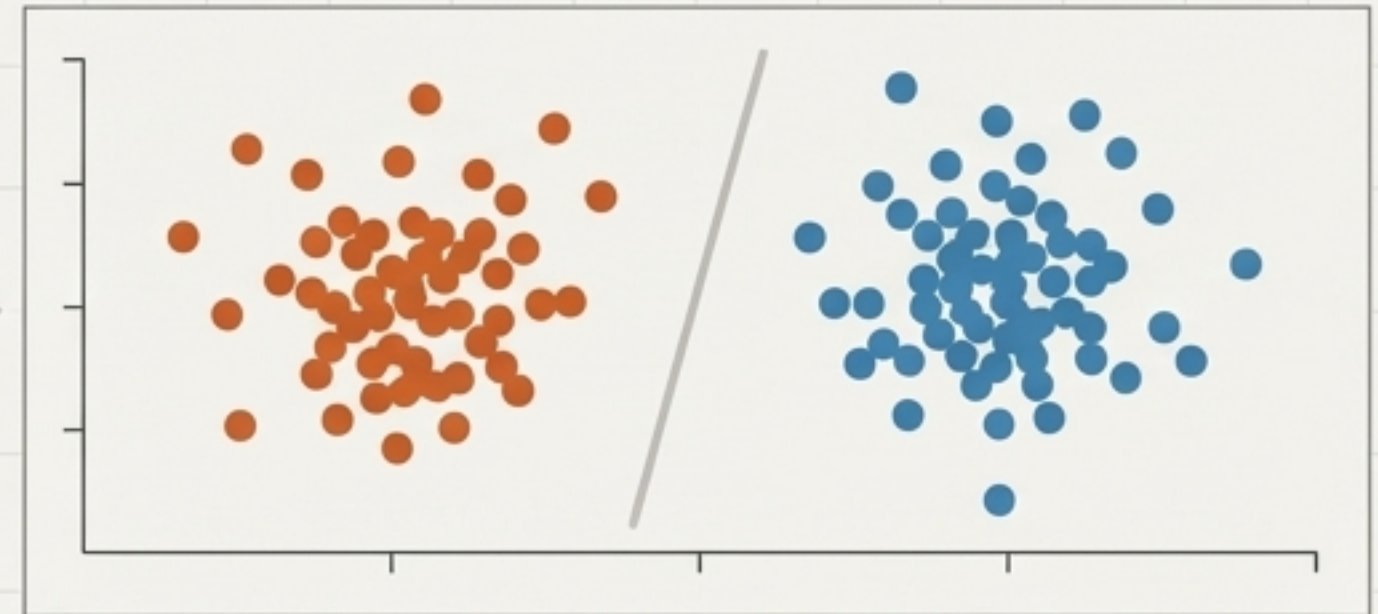
`max_iter = 1`

A single iteration leaves centroids trapped near their initial random sampling coordinates. The boundaries overlap.

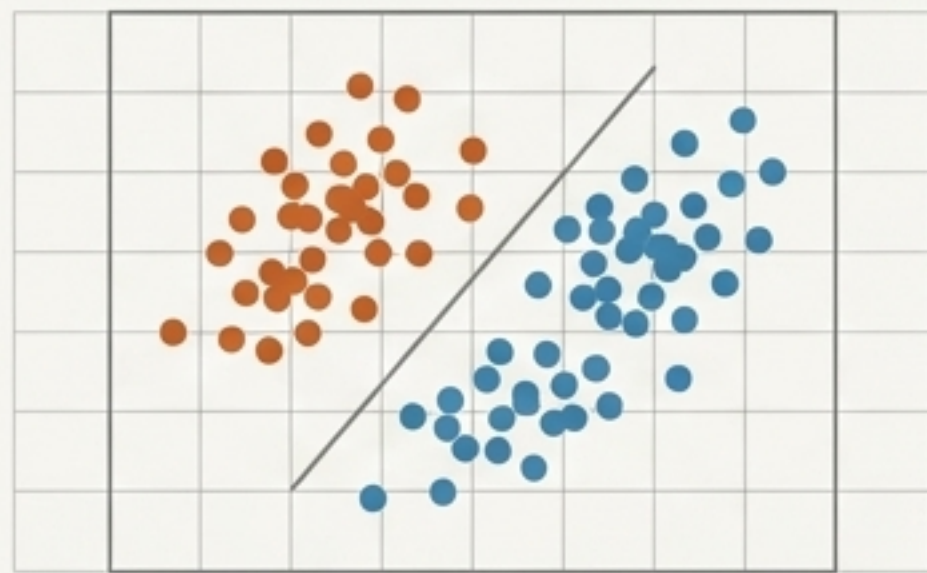


`max_iter = 100`

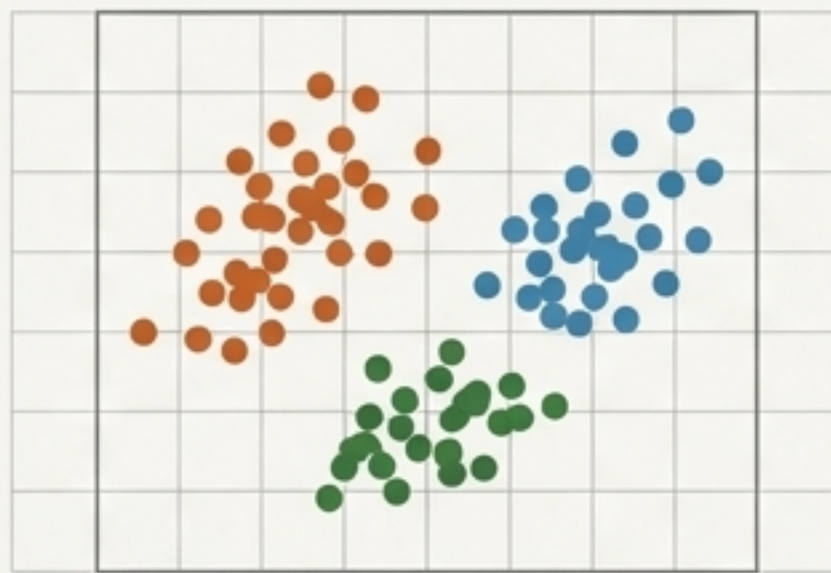
Sufficient iterations allow the algorithm to iteratively correct itself, settling into the true mathematical centres.



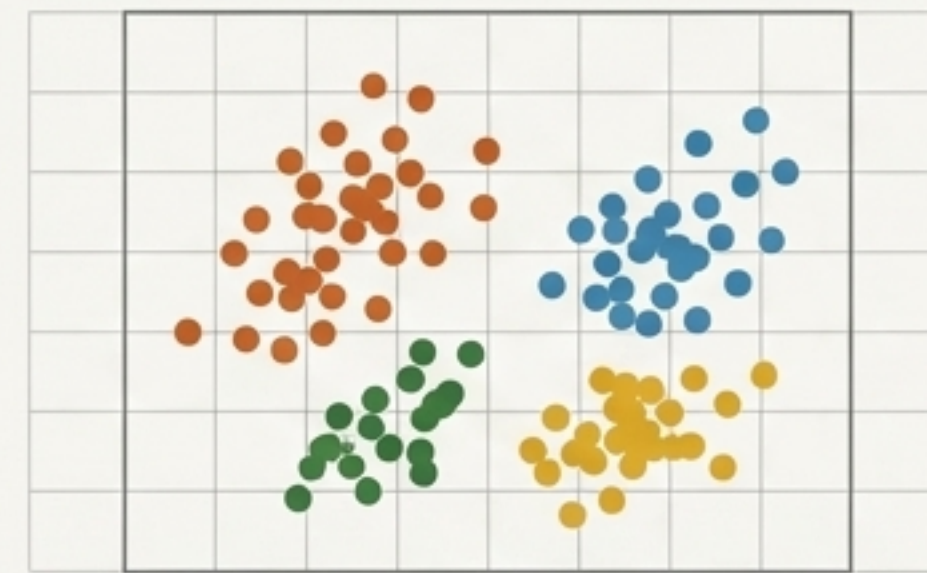
Dynamic Architecture Scales to Real-World Data



k=2



k=3



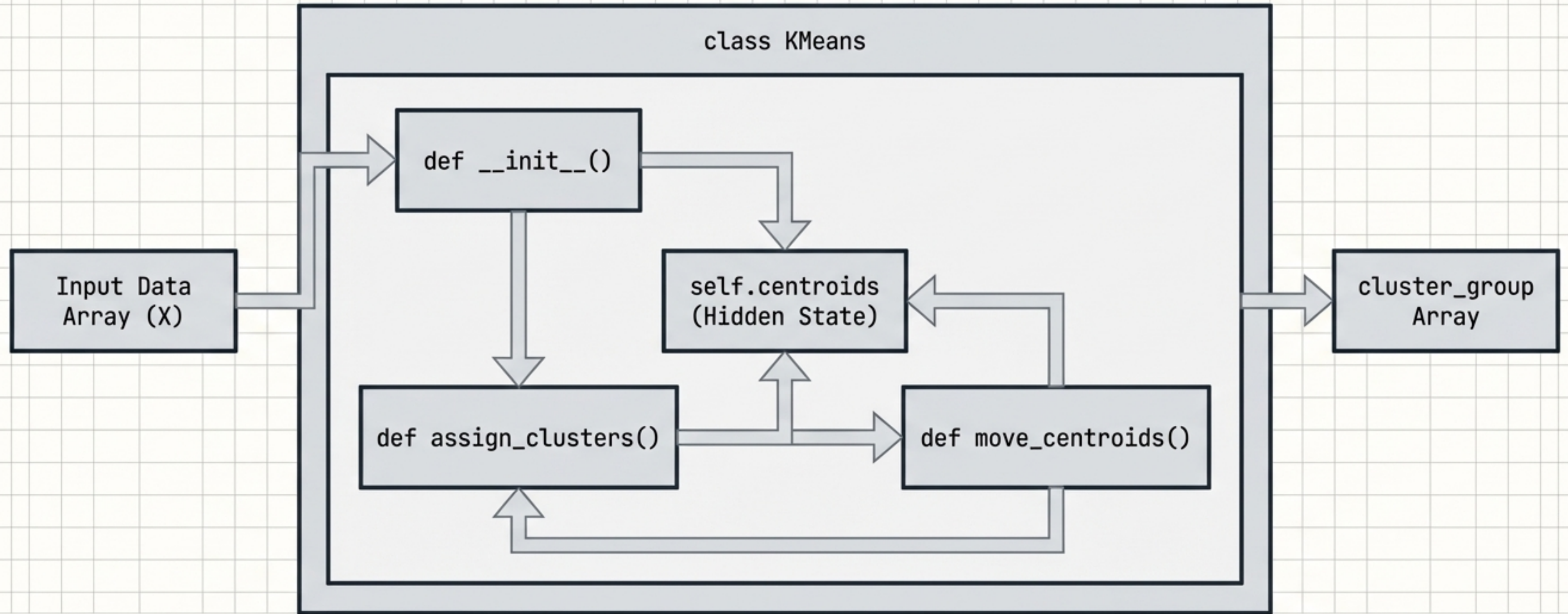
k=4

Math_Score	Reading_Score	IQ
64	64	130
43	90	130
66	64	150
39	88	130
68	45	120
72	72	130

```
data = pd.read_csv('student_clustering.csv')  
km = KMeans(n_clusters=4)  
km.fit(data)
```

Seamlessly handles
dynamic features and
undefined data.

Synthesis: The Complete Data Flow Architecture



Isolated methods mean nothing alone. True algorithmic power emerges when the `fit()` loop orchestrates the cycle, passing state until convergence.

Next Steps: Programmatically Finding K



The Elbow Method

We have built the core algorithm, but we currently hardcode the number of clusters.

The next engineering challenge is implementing Within-Cluster Sum of Squares (WCSS) to mathematically deduce the optimal clusters for any unknown dataset.